SRS

Smart Campus Assistant

אפליקציית רשת

<u>פרוייקט האקטון</u>

נכתב על ידי רון ספיר

8.3.2026

הרקע

במסגרת תוכנית הכשרת הלוחמים "לוחמים להייטק" הגענו לשלב ההאקטון, שהוא אירוע יצירה מרוכז ואינטנסיבי, בדרך כלל בן 24-48 שעות, שבו צוותים מפתחים פתרונות טכנולוגיים או רעיוניים לבעיות מוגדרות.

בכורח הנסיבות, השתנה ההאקטון וקיבלנו פרויקט בן 5 ימים שבו אנחנו צריכים לפתח מערכת עזר ווביתחכמה לסטודנטים.

הצורך

מערכת שתעזור לסטודנטים לקבל תשובות לשאלות נפוצות בנוגע למידע קמפוסי.

מטרות המערכת:

- **מטרת העל** - לייעל את התקשורת בין הסטודנטים לקמפוס
- להקל על הסטודנטים ולאפשר להם לחסוך זמן יקר
- לאגד ולנהל את המידע הקמפוסי
- לאפשר לתלמידים גישה לאזור נוח עם מידע מהיר ואמין
- להראות שהקמפוס מודרני ומחובר לטכנולוגיות אקטואליות

הפתרון – Smart Campus Assistant

SCA היא אפליקציית web שנועדה לייעל את התקשורת בין הסטודנטים למוסד האקדמי בו הם לומדים, המערכת תאפשר לסטודנטים לשאול שאלות טקסט בנושאי קמפוס כגון שעות קבלה, לוחות מבחנים, שאלות נפוצות, דרכי הגעה ועוד. ולקבל תשובות מהירות ומדויקות בעזרת בינה מלאכותית שתעשה את האינטגרציה הנדרשת מול מסד הנתונים של הקמפוס.

מי הם בעלי העניין ? (Stakeholders)

בעלי עניין ראשיים

הסטודנטים- אלה הם משתמשי הקצה של המערכת. השימוש באפליקציה יעניק להם שירות ומענה מהיר, אמין וזמין בכל זמן, ללא צורך בהמתנה למענה אנושי, 24/7. בנוסף, האפליקציה תאפשר להם לקבל מידע מדויק במקום אחד, בצורה נוחה ופשוטה, וכך תחסוך זמן ותייעל את ההתנהלות שלהם מול הקמפוס.

סגל הקמפוס- סגל הקמפוס ייהנה מהפחתה משמעותית בעומס הפניות מצד הסטודנטים, שכן האפליקציה תספק מענה מהיר ואוטומטי לשאלות נפוצות. כך יוכלו אנשי הסגל להתמקד בבעיות מורכבות ומשמעותיות יותר, תוך ניצול יעיל ומקסימלי של זמנם לשיפור השירות והתהליכים בקמפוס.

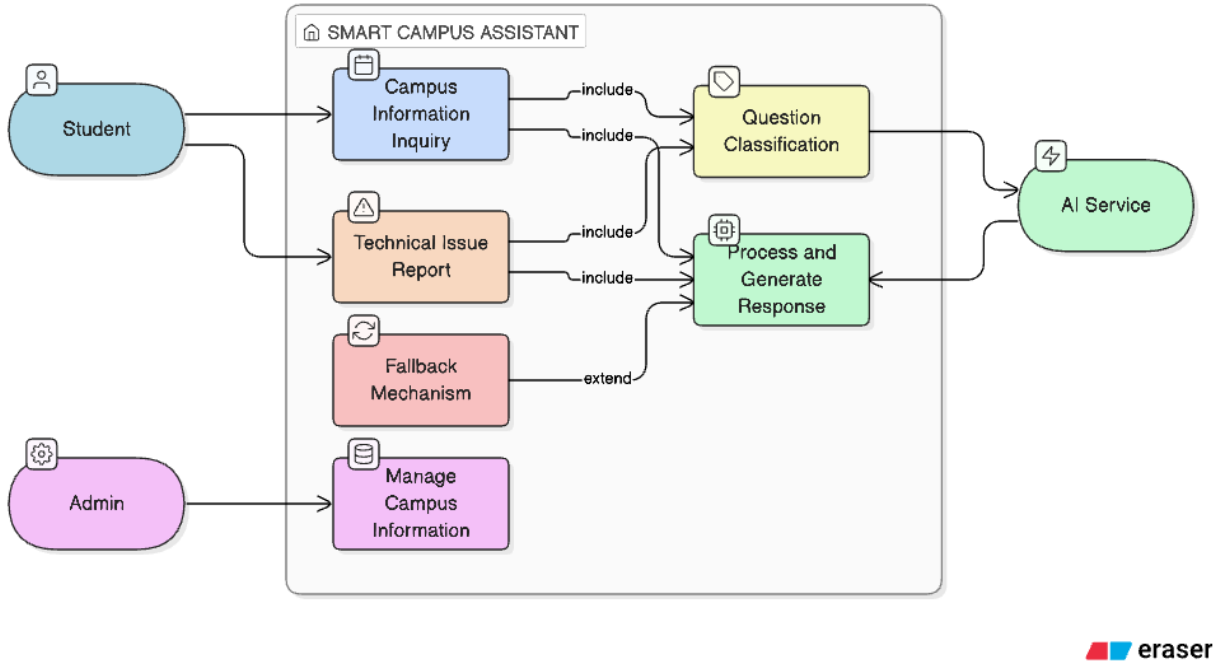
בעלי עניין משניים

הנהלת הקמפוס- כאשר הקמפוס מציג אפליקציה כזו לציבור הסטודנטים, הוא משדר חדשנות, קדמה ומחויבות אמיתית לרווחת הסטודנטים שלו. מהלך כזה מחזק את תדמית המוסד כמוסד מודרני המאמץ טכנולוגיות מתקדמות לשיפור חוויית הלמידה והשירות. כתוצאה מכך, שביעות הרצון של הסטודנטים עולה, דבר התורם לחיזוק המוניטין של הקמפוס, להגדלת האטרקטיביות שלו בקרב מועמדים חדשים, ובסופו של דבר גם לעלייה בביקוש ללימודים ובהכנסות המוסד.

צוות הפיתוח- הצלחת הפרויקט יכולה במידה רבה להוביל להרחבת השימוש במוסדות נוספים, להגדיל את החשיפה של החברה בקרב המוסדות ובכך להביא לעבודה והכנסה נוספת למפתחים.

תרחישי שימוש (use cases)

להלן דיאגרמה המציגה את כלל שחקני המערכת ואת הuse cases שיוצגו בהמשך.



Use case 1: בירור מידע מבוסס זמן

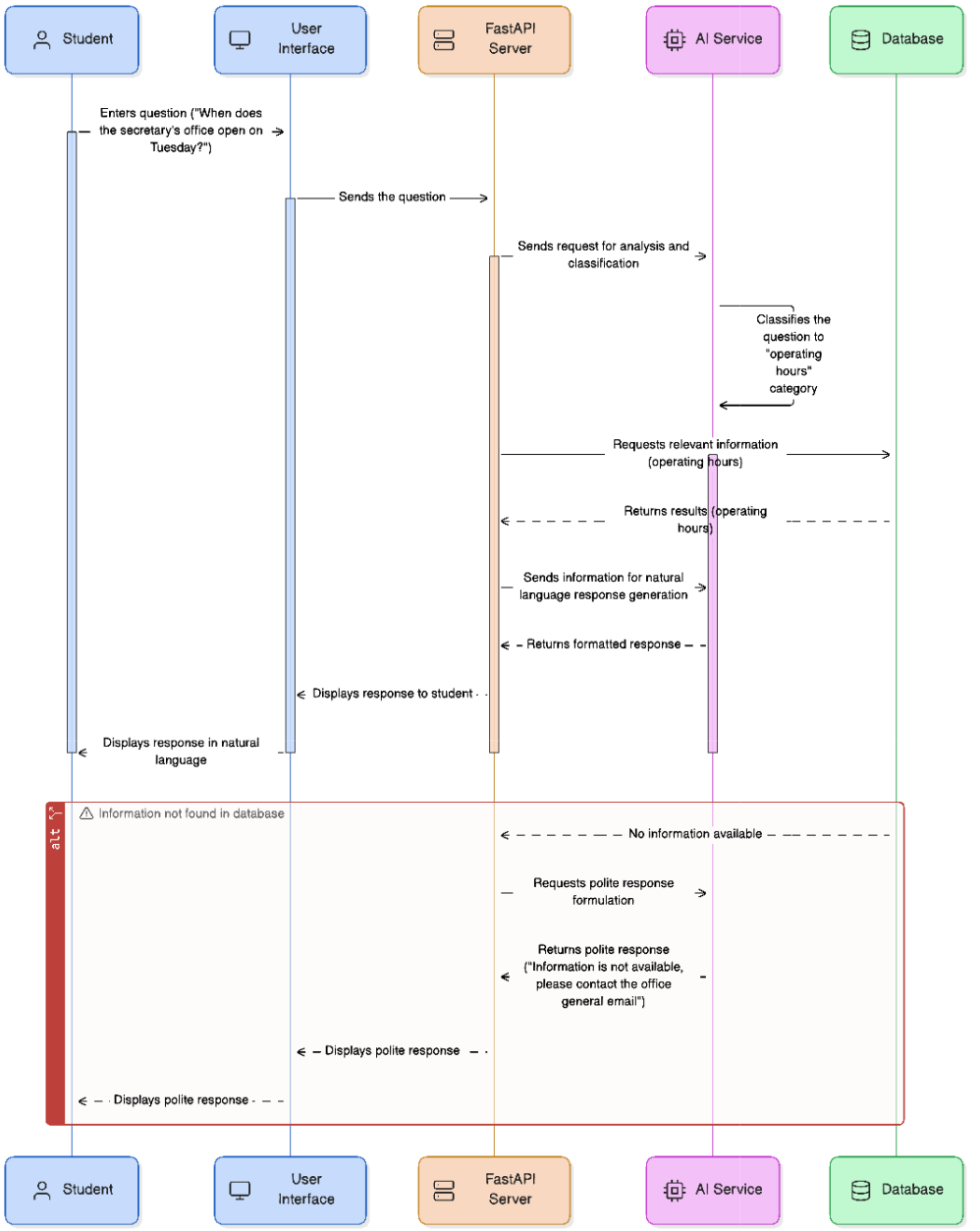
שחקן ראשי: סטודנט

תיאור: הסטודנט מעוניין לדעת מתי משרד המזכירות נפתח ביום מסוים

זרימת מידע: מוצגת בדיאגרמה

Edge case: המידע המבוקש לא קיים במסד הנתונים (alt בדיאגרמה).

פתרון: המערכת תחזיר תשובה אדיבה המציינת ששעות הקבלה המדויקות אינן זמינות ותציע את המייל הכללי של המזכירות.



eraser

Use case 2: איתור מידע (מידע כללי)

שחקן ראשי: סטודנט

תיאור : הסטודנט מחפש את הכתובת של הקמפוס

זרימת מידע:

1. הסטודנט שואל: "איפה נמצאת מעבדת המחשבים של שנה א'?".
2. השרת מעביר את הבקשה והai מסווג אותה בקטגוריית "מידע כללי".
3. הai מנסח הכוונה מדוייקת בהתבסס על מידע מובנה שהוזן מראש.
4. התשובה מוחזרת ומוצגת לסטודנט.

Edge case: השאלה מנוסחת עם שגיאות כתיב קשות.

הפתרון: מודל ה-AI בנוי להתגבר על שגיאות קלות, אך אם הוא לא מצליח להבין את הכוונה, יופעל מנגנון הגיבוי ויבקש מהמשתמש לנסח מחדש.

use case 3 : דיווח על תקלות (בעיה טכנית)

שחקן ראשי: סטודנט

תיאור: הסטודנט נתקל בציוד תקול בקמפוס ורוצה לדווח עליו.

זרימת מידע:

1. הסטודנט כותב: "המקרן באולם x לא נדלק".
2. הai מזהה את מילות המפתח ומסווג כקטגוריית "בעיה טכנית".
3. השרת מחזיר תבנית תשובה שקבועה מראש לכל תקלה/ מקרה חירום.

Edge case: סטודנט מדווח על תקלה קריטית (למשל, הצפה). המערכת יכולה לזהות מילות חירום ולהדגיש את מספר הטלפון של התורן.

use case 4: חוסר ודאות של מודל השפה (Fallback Mechanism)

שחקן ראשי: סטודנט

תיאור: הפעלת מנגנון גיבוי כאשר המערכת לא בטוחה בתשובה שלה.

זרימת המידע:

1. הסטודנט שואל שאלה מורכבת או מבלבלת.
2. שירות הai מעבד את השאלה, אך הconfidence score שלו נמוך.
3. במקום להחזיר תשובה שגויה או הזיה, הai עוצר את הפקת התשובה.
4. המערכת מחזירה הודעה מובנת מראש למשל "אני עדיין לא יודע את התשובה לשאלה זאת. אני ממליץ לפנות למזכירות הקמפוס ומספר הטלפון ".

Use case 5: סינון תוכן והגנה על המערכת.

שחקן ראשי: ai service

תיאור: התמודדות עם משתמש שמנסה להשתמש בai שלא למטרתו העיקרית.

זרימת המידע:

1. הסטודנט מקליד: "תכתוב לי קוד בפייתון שממיין רשימה" או מתכון למאכל כלשהו.

2.בהתבסס הsystem prompt הנוקשה שהגדרנו בשלב הפיתוח, הai מזהה חריגה של השאלה מגבולות המידע הקמפוסי.

3. הai מסרב בנימוס לענות ומחזיר את המשתמש להקשר לדוגמה- "אני העוזר החכם של הקמפוס ונועדתי לעזור בשאלות אקדמיות ומנהלתיות בלבד. איך אוכל לעזור לך בנושאי הקמפוס?"

Use case 6: ניהול, עדכון והעשרת הdb

שחקן ראשי:מנהל המערכת (admin)

תיאור: תהליך הוספת נתונים חדשים או עדכון מידע קיים (כגון שעות פעילות, אנשי קשר או נהלים חדשים בקמפוס), במטרה להבטיח שמנגנון ה-RAG של הבינה המלאכותית מספק לסטודנטים מידע אמין, רלוונטי ועדכני.

תנאי מקדים: לאדמין חייבת להיות גישה לbackend ולdata base

זרימת המידע:

1. האדמין מאתר צורך בעדכון נתונים (למשל: שינוי שעות קבלת קהל במזכירות עקב תקופת מבחנים).ניגש לקובץ הdb שלו.

2. האדמין מזין את הרשומות החדשות, מעדכן את הרשומות הקיימות, ומבצע שמירה לdata base.

3. הנתונים נשמרים בהצלחה וזמינים באופן מיידי לשכבת שליפת הנתונים.

4. החל מהרגע שנשמר, הסטודנטים יוכלו לקבל מידע עדכני ורלוונטי בfront.

דרישות פונקציונליות:

1. האפליקציה תקבל endpoint שתקבל את השאלות בטקסט חופשי.

2. האפליקציה תבצע קטגוריזציה של השאלה שמתקבלת לאחת מ3 קטגוריות לפחות לדוגמא- לוח זמנים , מידע כללי, בעיה טכנית.

3. המערכת תשלב ותשתמש בapi לעיבוד השאלה.

4. המערכת תכלול מנגנון fallback בסיסית לתשובות שגויות.

5. המערכת תכלול מנגנון rate limiting שתחזיר למשתמש הודעה מובנת מראש במקרה שהוא ישלח כמה שאלות בבת אחת.

דרישות לא פונקציונליות:

1. ביצועים וזמן תגובה של האפליקציה- התגובה לא תעלה על 5 שניות ב95% מהמקרים. בעזרת ניהול בקשות בצורה א-סינכרונית.

2. אבטחה- מפתחות הapi שלנו ככלל ישמרו מאובטחים בקובץ .env

3. עמידות ואמינות של האפליקציה- בגלל שאנחנו תמיד מניחים שהמשתמש יעשה טעויות. האפליקציה תהיה עטופה בtry except ומנגנון fallback כדי למנוע קריסות.

4. אופטימיזציית תשתית- גודל הDocker Image לא יעלה על 250mb בעזרת כתיבת Dockerfile רב-שלבי (multi-stage) שמבודד את סביבת הבנייה מסביבת הריצה ומקטין משמעותית את גודל התדמית הסופית.

Backend Edge cases:

תרחיש 1: הai החיצוני קורס או לא מגיב.

הפתרון: הbackend יתפוס את השגיאה לפני שהמערכת תקרוס , ויפעיל מנגון הגנה שיחזיר לfront אל המשתמש תשובה מסודרת וקבועה מראש. לדוגמה "עוזר הai נמצא תחת עומס כרגע, אנא נסו פעם נוספת בעוד מספר דקות."

תרחיש 2: קלט זדוני או ריק מהמשתמש, כלומר המשתמש שולח טקסט ארוך (תהיה הגבלה של עד 300 תווים), ריק, או איזשהו קוד זדוני.

הפתרון: הbackend יבצע ולידציה מיידית לקלט, ויחזיר שגיאה למשתמש, פה המידע לא יעבור לאינטגרציית הai על מנת לייעל עלויות וזמן עיבוד.